OPCODA: SÍNTESE GRANULAR EM TEMPO REAL A PARTIR DA ANÁLISE ESTÁTICA DE BINÁRIOS EXECUTÁVEIS

Mateus Costa de Oliveira Santos

Rafael Augusto Garcez Elisei

Prof. Me. Warner Brezolin (orientador)

RESUMO

Este artigo apresenta o Opcoda, instrumento virtual que converte arquivos executáveis no formato Portable Executable em material sonoro por síntese granular em tempo real. O projeto parte de uma lacuna da música eletrônica e da arte experimental: o databending é praticado por importação de dados brutos e exportação de áudio, e os sintetizadores granulares comerciais operam sobre amostras WAV. O objetivo geral é um aplicativo autônomo e um plugin VST3 para Windows x64 que leia o executável diretamente, com a estrutura PE e a entropia por janela controlando os parâmetros granulares, abertura por diálogo ou arrasto, controle por MIDI e reprodução sem interrupções. A justificativa reúne três aspectos: a arquitetura sem trava e a leitura segura de binários, a acessibilidade da interface segundo as diretrizes WCAG 2.2, e o interesse dos autores em computação musical. As bases teóricas reuniram Pietrek e Sikorski e Honig sobre o formato PE, Shannon e Lyda e Hamrock sobre entropia, Roads, Smith e Cascone sobre síntese granular e processamento digital de sinais, e Pirkle sobre plugins de áudio. A metodologia caracterizou-se como pesquisa aplicada, com desenvolvimento de protótipo em etapas verificáveis. Os resultados incluíram a análise de três sistemas existentes, a modelagem do fluxo de dados, a auditoria de acessibilidade da documentação, a especificação da bateria de testes e a implementação do núcleo verificada por 191 casos de teste e casos de guard de alocação. Concluiu-se que a combinação de leitura verificada do formato PE, entropia como modulação e motor granular não foi encontrada nos sistemas comparados, e que a implementação é viável com a toolchain disponível. A validação de latência permanece pendente de estação de trabalho de áudio licenciada.

Palavras-chave: Síntese granular. Databending. Tempo real. Acessibilidade. VST3.

ABSTRACT

This paper presents Opcoda, a virtual instrument that converts Portable Executable files into sound through real-time granular synthesis. The work addresses a gap in electronic music and experimental art: databending is practised by importing raw data and exporting audio, while commercial granular synthesizers operate on WAV samples. The general objective is a standalone application and an x64 VST3 plugin that reads the executable directly, with PE structure parsing and windowed entropy driving the granular parameters, file opening via dialog or drag-and-drop, MIDI control and glitch-free playback. The justification covers the lock-free architecture and safe binary parsing, the WCAG 2.2 interface accessibility, and the authors' interest in digital audio. The theoretical bases included Pietrek and Sikorski and Honig on the PE format, Shannon and Lyda and Hamrock on entropy, Roads, Smith and Cascone on granular synthesis and digital signal processing, and Pirkle on audio plugins. The methodology was applied research with prototype development in verifiable stages. The results included the analysis of three similar systems, pipeline modelling, the accessibility audit of the documentation, the specification of the test battery and the verified implementation of the core, with 191 test cases and allocation-guard cases. The combination of verified PE parsing, entropy-driven modulation and granular engine was not found in the compared systems, and the implementation is feasible with the available toolchain. Latency validation remains pending a licensed digital audio workstation.

Keywords: *Granular synthesis. Databending. Real-time. Accessibility. VST3.*

1. INTRODUÇÃO

O databending nasceu de um gesto simples: abrir um arquivo com um programa que não foi feito para lê-lo e ouvir o que sai. A música eletrônica transformou esse gesto em repertório. O projeto alemão Oval riscou CDs com caneta permanente para amostrar o estalo do lazer de leitura e, com isso, fundou o gênero glitch (COOPER, 2024; RICHARDSON, 2015); Ryoji Ikeda, na série datamatics, promoveu os próprios dados a instrumento, convertendo erros de disco e código-fonte em som e imagem (IKEDA, 2006). Em ambos os

casos o material bruto do meio digital deixa de ser ruído a ser corrigido e passa a ser a fonte. A ferramenta que difundiu a prática foi a importação de dados brutos do Audacity (AUDACITY TEAM, 2026): declara-se codificação, canais, deslocamento e taxa de amostragem, e qualquer arquivo vira onda.

A onda, porém, chega morta na sessão. Não há polyphonia, não há resposta a nota, e o que se obtém é uma faixa estática, restrita ao que ficou gravado no disco; quem quisesse levá-la a um palco teria de importar o executável, exportar o áudio resultante e carregá-lo num sampler ou sintetizador granular, numa cadeia em que cada etapa congela o material um pouco mais. O próprio arquivo já impõe seus limites antes de qualquer edição: importado o Bloco de Notas do Windows a 44,1 kHz e 16 bits, o cabeçalho DOS abre a faixa com um salto de amplitude próximo da escala cheia, os zeros do preenchimento estendem um silêncio sem musicalidade alguma, e o segmento de código só então entrega o ruído denso que interessa.

Os sintetizadores granulares comerciais, por seu turno, pararam na outra margem. Um Granulator II ou um Quanta 2 oferecem tamanho de grão, densidade e dispersão com a naturalidade de quem espera uma amostra WAV por trás, mas não sabem ler estrutura de arquivo, não calculam entropia por janela, e ignoram que uma região do binário carrega assinatura estatística distinta da região vizinha. O Opcoda ocupa o espaço entre esses dois extremos: mantém os controles granulares que o produtor já conhece e acrescenta a leitura do formato PE, de modo que cada seção do executável vira uma fonte de grãos com posição, densidade e dispersão ajustáveis, entregue dentro do prazo de tempo real. O tema, assim, reúne três áreas que raramente aparecem juntas num trabalho de graduação: análise estática de binários, teoria da informação como sinal de controle e engenharia de software em tempo real, às quais se soma uma interface pensada desde o início para operação por teclado, foco visível, contraste mínimo e controle por MIDI, de forma que músicos com restrição motora ou visual também toquem.

O objetivo geral é o Opcoda como aplicativo autônomo e plugin VST3 para Windows x64, abrindo ou recebendo por arrasto arquivos executáveis, com parâmetros granulares controláveis por MIDI, redução do deslocamento de corrente contínua de ao menos 40 dB e reprodução sem interrupções. Para chegar lá, os objetivos específicos encadeiam-se do arquivo ao som: um parser do formato PE próprio, com verificação de limites e conversão para ponto flutuante normalizado; ingestão somente leitura, com diálogo e arrasto, cópia defensiva e erros tipados; fila sem trava entre interface e motor; motor granular de oito vozes com grãos de 1 a 100 ms e janelas Hann e Gaussiana; entropia de Shannon por janela e por seção como modulação; filtro de bloqueio de corrente contínua e limitador na saída; interface com mapa de seções, grelha navegável e controles automatizáveis; isolamento entre interface e motor, verificado por guard de alocação; e mapeamento MIDI com notas, pedal de sustentação e dois controladores de mudança de controle.

A pergunta de pesquisa acompanha essa cadeia: como construir um instrumento em tempo real que leia binários PE arbitrários com segurança e os converta em material sonoro controlável, por síntese granular guiada por entropia, sem travar o host de áudio. A hipótese H1 afirma que o pipeline reduz o deslocamento de corrente contínua em ao menos 40 dB, mantém o processamento abaixo de metade do orçamento de bloco com oito vozes e registra zero travamentos sob entrada malformada; a hipótese nula H0 nega diferença relevante ante a leitura bruta direta. O artigo percorre a fundamentação teórica, a metodologia e a arquitetura, os resultados da verificação automática e da avaliação com usuários, e a conclusão.

2. FUNDAMENTAÇÃO TEÓRICA

2.1 O formato PE e a leitura estática de arquivos não confiáveis

O formato Portable Executable é o formato de executáveis, bibliotecas e drivers do Windows em x86 e x64, derivado do COFF, com especificação mantida pela Microsoft (MICROSOFT, 2024) e anatomia documentada campo a campo nos dois artigos de referência de Pietrek (PIETREK, 1994; PIETREK, 2002),

leitura obrigatória para quem nunca abriu um executável por dentro. Do cabeçalho DOS, cujo deslocamento aponta para o cabeçalho PE, passando pela assinatura PE, pelo COFF com número de seções e características da imagem e pelo cabeçalho opcional com ponto de entrada e diretórios de dados, até a tabela de seções e o corpo das seções, tudo é matéria-prima navegável; cada entrada da tabela declara nome, tamanhos virtual e em disco, ponteiro para os dados e flags de permissão que distinguem código, leitura e escrita, e o tamanho em disco e o tamanho em memória divergem tanto de forma legítima, como em dados não inicializados, quanto por corrupção, o que obriga toda leitura indexada por campo de cabeçalho a verificar limites antes de acontecer. Como os tamanhos divergem, as seções convencionais carregam perfis estatísticos próprios, e é exatamente essa segmentação que Sikorski e Honig tomam como ponto de partida da análise estática, que inspeciona o artefato sem executá-lo (SIKORSKI MICHAEL; HONIG, 2012); no Opcoda, cada seção vira uma região de textura estatística sonificável.

Seção	**Conteúdo típico**	**Perfil estatístico esperado**
.text	Código de máquina	Entropia intermediária a alta
.data e .rdata	Dados e constantes	Entropia mista
.rsrc	Ícones, bitmaps, diálogos	Regiões homogêneas e periódicas
.bss	Dados não inicializados	Entropia nula, sem dados brutos

Tabela 1: Seções PE e perfil de conteúdo esperado
Autores

A leitura segura aplica o princípio do menor privilégio: o arquivo é aberto em modo somente leitura e nenhuma página recebe permissão de execução. A execução acidental é eliminada por construção, e o binário nunca é executado, mapeado como executável ou desmontado. Um segundo princípio costuma ser tratado como detalhe e decide a sobrevivência do host: o tamanho declarado dos dados brutos de uma seção não é confiável. A única postura segura é tratá-lo como teto e recortá-lo para o que existe no arquivo (KLEIDERMACHER DOMINIK; KLEIDERMACHER, 2012).

2.2 Entropia de dados arbitrários

Shannon definiu a entropia de uma fonte discreta como a quantidade média de informação por símbolo (SHANNON, 1948):

$$H(X) = -\sum_{i} p(x_i) \log_2 p(x_i) \tag{1}$$

Aplicada a fluxos de bytes, a fórmula ganha contorno físico imediato: o alfabeto tem 256 símbolos e a entropia máxima é 8 bits por byte, atingida quando todos os valores são equiprováveis. Dois extremos calibram a leitura, porque um bloco de 1024 bytes idênticos produz entropia zero, já que um único símbolo concentra toda a probabilidade, enquanto um bloco com os 256 valores igualmente distribuídos atinge o teto teórico; entre esses polos fica o código compilado, com sua distribuição enviesada de opcodes e operandos, e a ele se acostam os blocos comprimidos ou cifrados, que se aproximam do teto com comportamento análogo ao ruído de banda larga. Lyda e Hamrock mostraram que esse perfil entrópico distingue classes de conteúdo em binários (LYDA RICHARD A.; HAMROCK, 2007), e o que a análise forense usa para separar código de payload cifrado é a mesma informação que a síntese pode usar para separar textura estável de textura instável.

Para que a informação vire controle, porém, um número só não basta: medir o arquivo inteiro devolve uma média que não serve para navegar, e por isso o Opcoda usa janela deslizante de 2048 bytes, o que produz uma curva ao longo do arquivo. É essa curva que modula a síntese, ponderando a dispersão temporal dos grãos, a aleatoriedade de posição e a densidade da nuvem, de modo que região de alta entropia gera textura instável e ruidosa enquanto região de baixa entropia gera textura estável e periódica; a ponte entre os dois

domínios é natural, porque a entropia resume em um escalar por janela a mesma distinção entre ordem e ruído que a granulação manipula no tempo.

A implementação dessa leitura custou um bug que passou em teste, e o bug ensinou o formato. Medir a entropia de uma seção não é medir a partir do deslocamento dela, porque o fim da seção é o deslocamento mais o tamanho, e esse tamanho é o mínimo entre o declarado e o disponível; uma seção de 256 bytes de zero seguida de dados uniformes mede cerca de 4,98 bits por byte quando medida a partir do deslocamento, e zero quando medida com comprimento explícito. A segunda é a resposta certa, e a diferença só apareceu quando um executável mínimo válido passou a ser construído em memória pelo próprio conjunto de testes.

2.3 Síntese granular e condicionamento de sinal

Roads formalizou a síntese granular a partir do conceito de microsom: o som como agregado de microfragmentos de 1 a 100 ms, cada grão com duração, envelope, frequência, amplitude e posição próprios, e com eles os parâmetros canônicos que todo granular expõe, de tamanho de grão a densidade, dispersão, afinação e panorâmica (ROADS, 2001). O formato granular, porém, pressupõe uma fonte cooperativa, e bytes de programa são tudo menos isso: interpretados como amostras, têm média não nula, descontinuidades nos limites de cabeçalhos e variações acima do limite de Nyquist, que se dobram para o espectro audível como aliasing (SMITH, 2007; ZÖLZER, 2011). O janelamento é a primeira resposta, porque multiplicar cada grão por uma função que zera as bordas elimina o clique por descontinuidade de fase na sobreposição, para janela de N amostras com n de 0 a $N-1$:

$$w(n) = 0{,}5\left(1 - \cos\frac{2\pi n}{N-1}\right) \tag{2}$$

A escolha de Hann e gaussiana no MVP segue esse mesmo compromisso, pois janelas suaves reduzem o espalhamento dos transientes ao custo de resolução temporal, e Hamming e Blackman (SMITH, 2007) ficam documentadas para uso futuro. Janela nenhuma, contudo, resolve o deslocamento de corrente contínua, que sai por um filtro passa-alta de primeira ordem com a recorrência e o coeficiente de polo abaixo (SMITH, 2007):

$$y[n] = x[n] - x[n-1] + R\,y[n-1] \tag{3}$$

$$R = \exp\left(-\frac{2\pi f_c}{f_s}\right) \tag{4}$$

Com $R = 0{,}9983$ e $f_s = 44{,}1$ kHz, a frequência de corte fica em torno de 12 Hz, dentro da faixa de 10 a 15 Hz que preserva o conteúdo audível; com $R = 0{,}995$, o mesmo cálculo dá 35 Hz, que come parte do grave que o instrumento deveria preservar. A divergência entre os quatro documentos que citavam 0,995 e o corte que eles prometiam apareceu durante a implementação, e está registrada aqui porque o ensaio espectral mede a faixa de frequência e não o coeficiente: corrigidos os textos, a cadeia fecha com biquad ressonante na forma direta (ZÖLZER, 2011) e limitador de teto a -1 dBFS na saída.

O resultado é uma leitura particular da estética que Cascone descreveu, em que artefatos digitais deixam de ser defeito e viram vocabulário composicional (CASCONE, 2000): o Opcoda segue essa linhagem com a diferença de que o glitch permanece fonte timbrística, mas sob controle paramétrico e dentro de garantias de tempo real.

2.4 Tempo real rígido em plugins de áudio

Formatos profissionais impõem contrato rígido ao callback de processamento, cujo período por bloco é da ordem de milissegundos (STEINBERG, 2024; CLAP, 2024; PIRKLE, 2019), e do contrato decorrem três proibições, todas observáveis: nenhuma alocação dinâmica durante o processamento, nenhuma entrada e saída ou chamada de sistema bloqueante no laço prioritário, e nenhuma primitiva de sincronização com espera. A última é a mais traiçoeira, porque se a thread de áudio, de alta prioridade, esperar um mutex detido pela interface, de baixa prioridade, o prazo do bloco passa a depender do escalonador, e um instrumento que depende do escalonador não é um instrumento de tempo real.

A resposta canônica separa dois domínios temporais: a interface roda a cerca de 60 quadros por segundo e admite alocação, entrada e saída e parsing, enquanto o motor de áudio opera só sobre memória pré-alocada, e a comunicação entre os dois viaja por fila circular sem trava de produtor único e consumidor único, com operações atômicas de ordenamento acquire-release e parâmetros suavizados dentro do próprio motor (PIRKLE, 2019).

Buffer em amostras	**44,1 kHz**	**48 kHz**	**96 kHz**
128	2,9 ms	2,7 ms	1,3 ms
256	5,8 ms	5,3 ms	2,7 ms
512	11,6 ms	10,7 ms	5,3 ms

Tabela 2: Orçamento de tempo disponível por bloco nas configurações do plano de testes
Autores

Declarar que o motor não acessa o estado da interface é apenas uma intenção; o que a torna verdade é não existir caminho de código que o permita, somado a um teste que falha se alguém criar esse caminho, e é por isso que o projeto invoca a biblioteca de framework de áudio apenas no adaptador e mantém o núcleo livre dela (JUCE, 2026).

3. METODOLOGIA E ARQUITETURA

3.1 Abordagem e etapas

A pesquisa é aplicada, com desenvolvimento de protótipo e revisão bibliográfica, e caminha em duas frentes de avaliação: na engenharia, mede-se se o sistema cumpre contratos verificáveis; com usuários, mede-se o uso musical do instrumento, o que está descrito na seção 5. O desenvolvimento seguiu especificação antes de implementação, com uma constituição de projeto que define seis portões, de build a documentação, pelos quais nenhuma tarefa fechou sem passar, e com rastreabilidade explícita nos dois sentidos entre objetivo, portão e ensaio. As etapas se distribuíram em cinco semanas, e a ordem inverteu o plano original, que posicionava a interface depois do MIDI, porque sem um plugin que abra no host não há como validar arrasto, parâmetros nem MIDI.

Etapa	**Foco**	**Entrega**
S1	Esqueleto e parser do formato PE	Núcleo C++20, parser com verificação de limites, suíte de testes
S2	Motor granular, arrasto e filtro de bloqueio	Motor de oito vozes, ingestão por arrasto, cadeia de condicionamento
S3a	Notas MIDI, estado e guardas	Gate por nota, pedal de sustentação, dois controladores, guard de alocação
S3b	Ensaio de latência no host	Bloqueada: exige DAW licenciado
S4a	Interface e grelha de bytes	Chassi de três faixas, controles, grelha, telemetria
S4b	Ensaio de aceite no host	Bloqueada: exige DAW licenciado
S5	Congelamento e validação	Auditoria de documentação e empacotamento

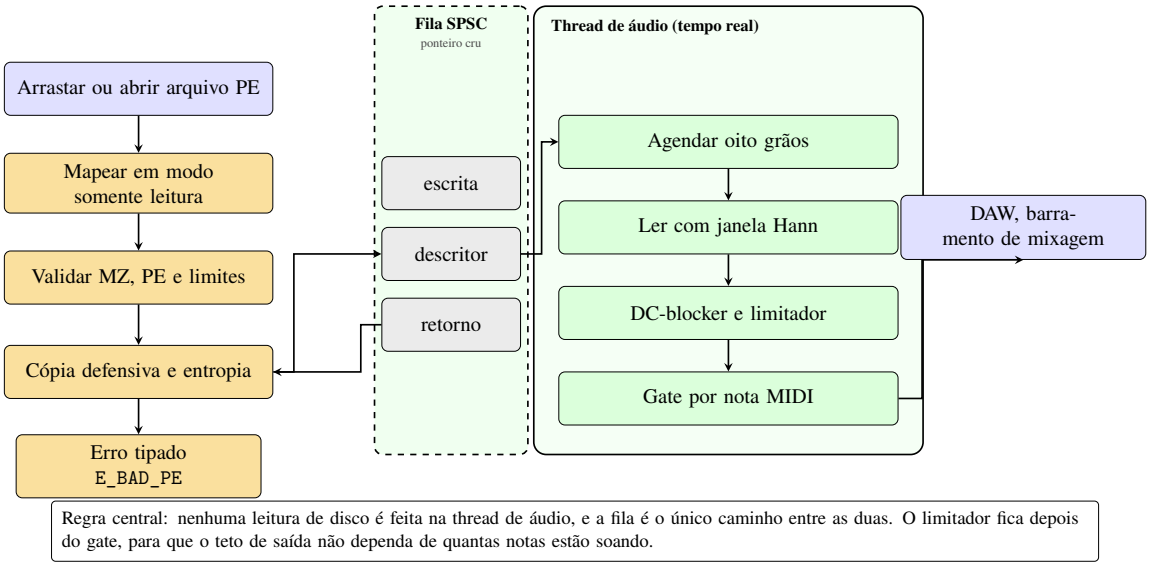


Figura 1: Pipeline de dados do Opcoda, da ingestão verificada à entrega no host
Autores

Tabela 3: Etapas de desenvolvimento e o que cada uma entregou
Autores

3.2 Pipeline e divisão de camadas

O projeto nasce de uma assimetria: a ingestão de dados arbitrários pode demorar e pode falhar, ao passo que a geração de áudio tem prazo de milissegundos e não admite falha, e a solução é separar as duas operações por uma fila, tornando essa separação o único caminho entre elas.

No lado que pode falhar, a ingestão e o parser rodam na thread de interface, abrindo o arquivo em modo somente leitura, validando assinatura e limites, copiando os dados para memória própria e calculando a entropia por janela e por seção; na fronteira fica a fila circular de produtor e consumidor único, mais os parâmetros atômicos; no lado que não pode, o motor roda na thread de áudio, agendando grãos, lendo a fonte com janela, misturando as vozes e aplicando o condicionamento, até a entrega na saída do host. A regra é que nada passa do parser para o motor sem cruzar a fila e o motor nunca chama o sistema operacional, e a regra vale porque o núcleo não conhece a interface, verificado por teste.

Essa verificação só é possível porque o código se divide em três partes: o núcleo, com mil setecentos e setenta linhas de C++20 e nenhuma dependência de framework, contendo parser, conversão de byte para amostra, entropia, motor granular, janelas, filtro, limitador, fila, rastreador de notas e guard de alocação; o adaptador, com cerca de três mil e duzentas linhas, contendo processador de áudio, formatos e interface, e sendo a única parte que conhece o framework; e os testes, que somam duas mil quinhentas e cinquenta linhas e exercitam o núcleo sem o framework no caminho. É essa divisão que permite o ensaio de ponta a ponta, percorrendo leitura de disco, parser, conversão, motor, filtro e limitador sem framework e sem host, e foi nele que os dois defeitos mais caros do projeto apareceram com arquivo de verdade, um deles pego pelo AddressSanitizer como leitura além do fim do buffer.

3.3 Parser e verificação de limites

O parser percorre o arquivo em quatro validações encadeadas, cada uma com código de erro próprio: a assinatura MZ é conferida antes de qualquer campo ser lido, porque todo o resto do formato a pressupõe; a

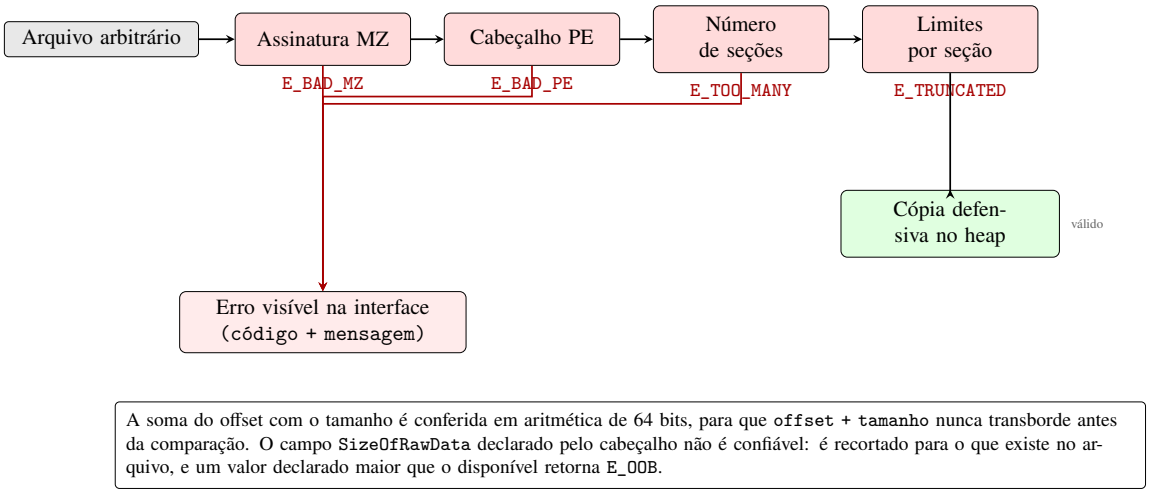


Figura 2: Cadeia de validação do parser e erros tipados correspondentes
Autores

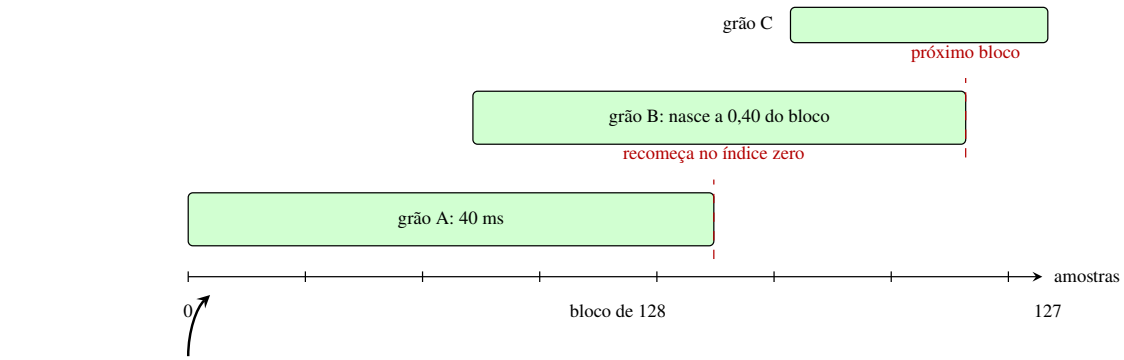
assinatura PE é conferida no deslocamento declarado pelo cabeçalho DOS, e esse deslocamento é validado contra o tamanho do arquivo antes de ser seguido; o número de seções é lido e limitado, e valor acima do limite é rejeitado em vez de alocar uma tabela do tamanho que o cabeçalho pedir; e cada seção tem deslocamento e tamanho conferidos contra o tamanho real do arquivo.

Abaixo das quatro validações, toda leitura passa por uma função única de verificação de limites, que confere a soma em aritmética de 64 bits para que deslocamento mais tamanho nunca transborde antes da comparação, e o tamanho declarado dos dados brutos, por não ser confiável, é recortado para o que existe; nenhum erro propaga exceção pela interface binária do plugin, e o material que estava em reprodução continua tocando. A robustez dessa cadeia só se provou porque a equipe construiu executável de teste em vez de confiar em arquivos reais, e um defeito concreto ilustra o porquê: o parser lia o número de seções no deslocamento mais dois a partir do cabeçalho PE, que pertence à assinatura de quatro bytes, e não no cabeçalho COFF, que começa no deslocamento mais quatro, o que produzia erro de assinatura em todo arquivo válido, com o erro no código e não na especificação do formato.

3.4 Motor granular e agendamento

O motor mantém oito vozes fixas, alocadas na preparação, cada uma com posição de leitura, taxa de avanço, índice de escrita, contador de amostra e fase de dispersão, e nenhum contêiner cresce durante o processamento. Três decisões tornam o agendamento previsível, e Figura 3 mostra o efeito: a densidade é um spawn por amostra derivado dos grãos por segundo, com fração de fase preservada entre blocos para que 128 e 512 amostras produzam a mesma densidade; o índice de escrita é posição dentro do bloco corrente e não contador global, e um grão que atravessa o bloco recomeça em zero, porque sem esse reset a voz trava e produz silêncio sem erro; e o avanço por amostra é a taxa de reprodução dividida pelo comprimento da fonte, já que a posição é normalizada entre zero e um.

A terceira decisão corrigiu um defeito silencioso que o critério de aceite não via: somando a taxa bruta, o grão varria o arquivo inteiro em duas amostras e morria, com pico de saída da ordem de dez elevado a menos sete, e o ensaio de atenuação continuava verde, porque uma asserção de valor maior que zero é satisfeita por um número desse tamanho; a correção foi tornar o agendamento observável por testes próprios, em vez de reclamar do critério. Fechado o agendamento, o interpolador é linear, com passo limitado para impedir aliasing audível na afinação máxima de mais vinte e quatro semitons, todo grão começa com a janela no índice zero, zerando a amplitude nas bordas, e a entropia local modula a dispersão, de modo que região



Cada grão guarda o índice de escrita onde nasceu, porque a sobreposição precisa começar na amostra certa. O índice é a posição dentro do bloco corrente, não um contador global: sem esse reset a voz trava e nunca mais escreve. Todo grão começa com a janela no índice zero, o que zera a amplitude nas bordas e evita o clique na sobreposição.

Figura 3: Agendamento dos grãos dentro de um bloco de 128 amostras
Autores

de alta entropia espalha mais e região repetitiva fica mais ancorada.

3.5 Fronteira de tempo real e interface

A troca de material usa duas filas em sentidos opostos, e não uma, porque o caminho de ida sozinho esconderia uma pergunta sem resposta: a interface publica o ponteiro do novo material e guarda o buffer em contêiner próprio, a thread de áudio troca o ponteiro ativo e devolve o antigo pela fila de retorno, e só então a interface remove o buffer, sabendo que a thread terminou de usá-lo. Uma fila só seria mais curta e incorreta, pois sem a volta a interface não sabe quando é seguro liberar, e liberar cedo é uso após liberação; e a fila carrega ponteiro cru, nunca ponteiro compartilhado, porque decrementar contador na thread de áudio pode rodar o operador de liberação ali dentro.

O guard de alocação transforma a restrição em código: o operador global de alocação é substituído por uma versão que conta alocações dentro da janela de áudio, e um teste marca a janela e falha se o processamento de bloco alocar, inclusive em cinco minutos de reprodução sintética. A lacuna que o guard não cobre é dita no texto: ele roda sobre o núcleo e não vê o processamento de bloco do plugin, onde as mensagens MIDI são lidas.

A interface segue a convenção do Ableton em três faixas, cabeçalho claro, display escuro e painel de parâmetros, e o motivo é funcional: halo de brilho sobre superfície clara desaparece, com contraste de 2,4 para 1, de modo que toda a energia de brilho fica na região escura. Os controles giratórios são componentes reais de deslize do framework e não desenho customizado, porque o componente entrega foco por teclado, ajuste com setas e manipulador de acessibilidade de graça, e o nome acessível de cada controle é a descrição completa em português, não o rótulo curto do protótipo.

O display é uma grelha de bytes no formato de um despejo hexadecimal, com endereço à esquerda, dezesseis bytes por linha e coluna de caracteres à direita.

O clique escreve o parâmetro de posição, porque escolher um byte é escolher de onde se lê. Se o byte estiver fora da região em reprodução, o editor puxa a região para lá antes de escrever o parâmetro, e a ordem não é arbitrária: a posição é fração da região, e escrevê-la antes de a região mudar apontaria para o sítio

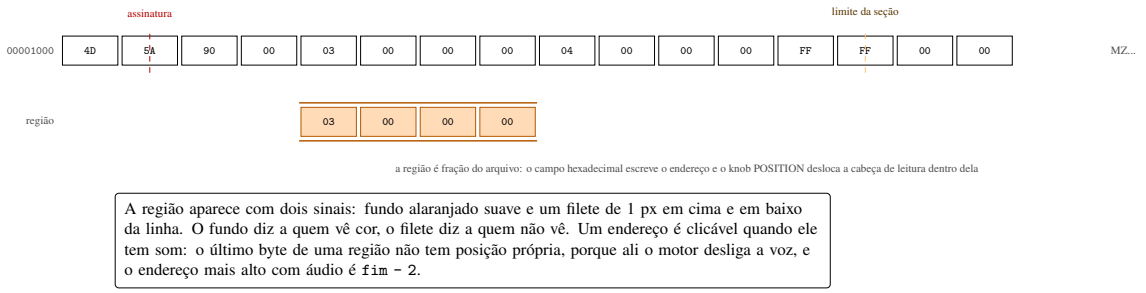


Figura 4: Grelha de bytes com a região de leitura em reprodução
Autores

errado.

O rastreio de notas separa teclas premidas de notas presas pelo pedal de sustentação, porque com o pedal premido largar o teclado não desliga o som, e soltar o pedal não abafa as teclas que continuam premidas. O gate entra antes do limitador, para que o comportamento do limitador não dependa de quantas notas estão soando, e a rampa é linear, de cinco milissegundos, porque abrir a saída de uma vez produz um degrau, e degrau é transiente largo em frequência.

Parâmetro	**Faixa**	**Controle**
Tamanho de grão	1 a 100 ms	Controle giratório, MIDI, automação
Densidade	1 a 200 grãos por segundo	Controle giratório, MIDI, automação
Posição	0 a 100 %	Grelha de bytes, MIDI, automação
Dispersão	0 a 100 %	Controle giratório, entropia local
Afinação	menos 24 a mais 24 semitons	Controle giratório, teclado MIDI
Volume e panorâmica	até 0 dB, esquerda a direita	Controle giratório, automação

Tabela 4: Parâmetros do MVP e o controle de cada um
Autores

3.6 Sistemas similares e acessibilidade

A comparação separa as duas metades do problema, e cada metade é habitada por ferramentas que ignoram a outra. De um lado, quem lê arquivo arbitrário: a importação de dados brutos do Audacity é a referência, aceita qualquer arquivo e produz faixa estática, sem polyphonia e sem parâmetros (AUDACITY TEAM, 2026), e a ela se somam objetos Max/MSP e utilitários de linha de comando que tampouco operam como instrumento. Do outro, quem faz síntese granular sobre amostras: Granulator II, de Robert Henke, é a referência do gênero, aceita qualquer amostra arrastada com até trinta e duas vozes e roda como dispositivo Max for Live (HENKE, 2013); Quanta 2, da Audio Damage, é a versão comercial multiformato (AUDIO DAMAGE, 2026); e ambos esperam WAV ou AIFF como fonte, sem fazer ideia da estrutura do arquivo, porque para eles o arquivo já é áudio.

Critério	**Databending em editor**	**Granular sobre amostras**	**Opcoda**
Fonte aceita	Qualquer arquivo	WAV e AIFF	Executáveis e PE
Polyphonia	Não	Sim	Sim, oito vozes
Resposta a nota MIDI	Não	Sim	Sim
Leitura da estrutura do arquivo	Não	Não	Sim, com verificação de limites
Entropia como controle	Não	Não	Sim, por janela de 2048 bytes
Saída do processo	Arquivo de áudio exportado	Áudio em tempo real	Áudio em tempo real
Formato	Editor	Max for Live, VST3 e outros	Autônomo e VST3

Tabela 5: Classe das ferramentas de databending, classe dos sintetizadores granulares e o Opcoda
Autores

Nenhum dos dois lados atravessa para o outro, e é essa a lacuna que o Opcoda ocupa: lê o executável, mede a entropia por janela e usa o resultado para modular a granulação, herdeiro do databending na fonte e

do granular na forma de tocar. A acessibilidade fecha a seção porque atravessa o projeto inteiro: a auditoria aplicada à documentação corrigiu links genéricos, aprovou a estrutura de cabeçalho das tabelas e fixou o idioma da página, e para a interface definiu critérios de teclado, foco visível, nome acessível, contraste e uso da cor (W3C, 2023; W3C, 2022).

3.7 Bateria de testes e limitações de ferramenta

Ensaio	Método	Critério de aceite	Situação
T1 espectral	FFT com 65536 pontos em três binários	Deslocamento atenuado em 40 dB; energia abaixo de 20 Hz menor que -60 dBFS	executado
T2 desempenho	Blocos de 128 a 512 amostras, uma a oito vozes	Percentil 99 abaixo de metade do orçamento, sem falhas	bloqueado
T3 robustez	50 mutações e 10 casos de borda	Sem falha de segmentação, cobertura de ramo acima de 70%	borda executado
T4 aceite	Arrasto, seis parâmetros, dois controladores, 2 min por formato	Som em menos de 2 s; controlador move o parâmetro	bloqueado

Tabela 6: Ensaios, método e critério de aceite

Autores

A distinção entre pendente e bloqueado organiza a tabela: pendente é trabalho agendado, bloqueado é trabalho que não pode ser feito com o que existe nesta máquina, e T2 e T4 estão bloqueados porque se medem dentro de uma estação de trabalho de áudio, cuja única cópia do Ableton presente não é instalação licenciada, o executável arranca e nunca abre janela utilizável, pois a ativação não está feita; trocar o host não afrouxou critério, e o número continua sendo percentil 99 abaixo de metade do orçamento, com zero falhas. À falta de host somam-se duas limitações de ferramenta: o Windows não distribui tempo de execução de ThreadSanitizer, nem no compilador da Microsoft nem no da LLVM, e a verificação de concorrência recai sobre o guard de alocação; e o MemorySanitizer, que detectaria leitura de memória não inicializada, hipótese que resta para explicar um defeito descrito na seção 4, não existe para esse compilador.

4. RESULTADOS DA VERIFICAÇÃO AUTOMÁTICA

4.1 Suíte de testes

A suíte de verificação tem 191 casos, mais os casos do guard de alocação, todos verdes em release, e a cobertura atravessa o projeto de ponta a ponta, do parser e da entropia ao motor granular, ao filtro, ao limitador, à fila e ao rastreio de notas. O ensaio que fecha a cadeia usa um executável real do sistema, sem framework e sem host: o arquivo de teste tem duzentos quilobytes, sete seções e byte médio de 96 na seção de código, o que corresponde a um deslocamento de menos 0,245 em amostra normalizada, material que o filtro de bloqueio tem o que remover e que a leitura sintética não reproduzia.

4.2 Ensaio espectral

O ensaio de atenuação cumpre o critério em três casos encadeados. O primeiro compara o bin de corrente contínua da saída tratada com o mesmo bin da leitura bruta, em um executável sintético com média distante de zero, e a asserção exige atenuação de ao menos 40 dB; o segundo exige que o pico abaixo de 20 Hz fique abaixo de menos 60 dBFS; e o terceiro isola a cadeia, enviando ao filtro um sinal puramente constante e medindo a energia da saída depois de um período de acomodação, que fica abaixo do limiar. O protocolo inclui um controle negativo, a mesma conversão de byte para amostra sem motor e sem condicionamento,

sem o qual a atenuação seria um número sem referência. O controle negativo também expôs um limite do próprio critério: sendo uma asserção de limite inferior entre dois sinais, ela é satisfeita por um sinal cujo pico é da ordem de dez elevado a menos sete, sem que exista som audível, e foi exatamente o que aconteceu com o defeito do avanço por amostra, corrigido tornando o agendamento observável por testes próprios em vez de reclamar de um critério de razão.

4.3 Ensaio de robustez

Os dez casos de borda do plano passam, cobrindo arquivo vazio, arquivo de um byte, deslocamento apontando fora do arquivo, número de seções acima do limite, tamanho declarado de dados brutos maior que o disponível, seções com deslocamento fora do arquivo, assinatura MZ válida com assinatura PE ausente, cabeçalho opcional inconsistente, entropia sobre seção sem dados e truncamento no meio de uma seção, todos devolvendo erro tipado, sem exceção e sem falha de segmentação. A robustez declarada apoia-se inteira nesses dez casos, porque o corpus de cinquenta mutações por inversão de bit ainda não existe e entra como casos determinísticos quando existir, e o portão de cobertura de ramo acima de 70% não tem medição automática, já que a ferramenta de cobertura não está disponível no toolchain escolhido.

4.4 Defeito conhecido e não corrigido

Um defeito visível no aplicativo autônomo permanece sem correção, e a honestidade manda registrá-lo: os seis parâmetros arrancam com valores que não são os padrões declarados, as caixas mostram 1 ms, cerca de 29 por segundo e 69 por cento, onde os padrões são 40 ms, 20 por segundo e 50 por cento, e a confirmação em compilação separada afasta qualquer componente de interface como causa.

A instrumentação por escrita em arquivo de texto em três pontos deu os seguintes valores normalizados:

Ponto	Tamanho de grão	Densidade	Posição
Fim do construtor do processador	0,393939	0,095477	0,500000
Entrada da ingestão	1,000000	0,060000	0,688000

Tabela 7: Valores dos parâmetros normalizados em três pontos do arranque

Autores

A instrumentação estreita a janela do defeito: na saída do construtor os valores estão certos, que é o que o mecanismo de parâmetro deve fazer, e à entrada da ingestão estão errados, sendo que a ingestão não escreve parâmetros, de modo que a escrita acontece no arranque do aplicativo, entre o construtor e a primeira ingestão, sem editor e sem temporizador. Os outros três parâmetros só parecem certos, pois têm padrão zero, e memória zerada sem inicialização devolve o mesmo valor por falta de outro.

O AddressSanitizer não acusa nada, e descartados por leitura, todos os candidatos estão protegidos: a fila tem índices limitados, o rastreador de notas verifica limites, os parâmetros atuais usam a leitura do valor bruto e a publicação de telemetria escreve quatro atômicos. O que resta é leitura de memória não inicializada, que o AddressSanitizer não vê e o MemorySanitizer veria, e como este não existe para o compilador da Microsoft no Windows, fechar a lacuna exige um depurador com pontos de observação no arranque, porque a correção por leitura de código, sem medir, trocaria o defeito por outro.

5. AVALIAÇÃO COM PRODUTORES MUSICAIS

A avaliação com produtores responde à pergunta que os ensaios automáticos não alcançam, porque atenuação e latência verificam a qualidade do sinal e o prazo de processamento, mas não dizem se alguém consegue

tirar som de um executável. Esta seção descreve o desenho do estudo, as perguntas e o procedimento de análise.

5.1 Objetivos do teste

Cada objetivo do estudo preenche uma lacuna do que a engenharia mediu até aqui. Queremos saber se a materialização do binário em som é controlável por um músico sem treino formal em síntese granular; se os parâmetros têm significado percebido, isto é, se mover densidade ou dispersão muda o som de forma reconhecível; quanto tempo o produtor leva até um som que considera útil, e se a interface de três faixas com MIDI ajuda ou atrapalha nesse percurso; e, por fim, quais preferências e sugestões orientam a próxima iteração.

5.2 Participantes e recrutamento

O estudo prevê cinco a oito produtores musicais com pelo menos dois anos de prática, e o critério de inclusão é familiaridade com DAW e com ao menos um instrumento virtual, não necessariamente granular; produtores iniciantes entram na amostra porque o instrumento atende justamente quem não tem biblioteca de amostras cara, e a análise separa os dois perfis para não confundir falta de habilidade com falta de usabilidade. O recrutamento combina convite direto nos grupos de produção musical da região com chamada nas redes sociais dos cursos de música e produção, e cada participante recebe um termo de consentimento que explica objetivo, gravação, tratamento de dados e caráter voluntário, com nenhum áudio publicado e liberdade de desistir a qualquer momento sem justificativa.

5.3 Materiais

O material do teste é o bundle VST3 mais o aplicativo autônomo, e cada participante recebe três executáveis de referência escolhidos por diversidade estatística, um executável de sistema pequeno, um binário comprimido com empacotador e um arquivo grande, seguindo o mesmo critério do ensaio espectral para que a diversidade do material não confunda a análise de preferência. Antes da sessão, o plugin é instalado numa máquina com Ableton Live licenciado, que é a mesma condição que bloqueia T2 e T4: sem host licenciado, nem a telemetria de latência nem o aceite visual do arrasto podem ser medidos dentro da DAW, e o estudo com usuários herda essa dependência.

5.4 Tarefas

Cada participante executa quatro tarefas em ordem contrabalançada, para evitar que o primeiro exemplo defina a expectativa do resto: no Carregamento, arrasta um dos três executáveis para o plugin e relata o tempo percebido até o primeiro som; na Exploração, percorre os seis parâmetros e produz dois ajustes que mudem o som de forma satisfatória; na Performance, toca por dois minutos com notas MIDI e pedal, mapeando ao menos um controlador de mudança de controle; e no Erro, carrega deliberadamente um arquivo inválido, como um arquivo de texto, e relata o que aconteceu e se a mensagem foi compreensível.

5.5 Instrumentos de coleta

A coleta combina quatro instrumentos que se corrigem entre si. A escala SUS de Brooke (BROOKE, 1996), dez itens de cinco pontos e nota de 0 a 100, mede a impressão global de facilidade e satisfação; um

questionário de atributos complementares mede se o produtor percebeu efeito audível em cada parâmetro; o diário de sessão, em que o produtor registra em uma linha o que esperava e o que ouviu a cada ajuste, capta a distância entre intenção e resultado; e a telemetria instrumentada no plugin, coletada em arquivo durante a sessão, registra vozes ativas, pico de saída e entropia da janela de leitura ao longo dos dois minutos, sem captura do áudio.

5.6 Perguntas de pesquisa

As perguntas foram escritas antes de qualquer sessão, e são as que a análise responde. Sobre controlabilidade: o produtor consegue obter, com os parâmetros do Opcoda, uma variedade de sons que ele reconheça como diferentes entre si? Sobre tempo: quanto tempo leva, em minutos, para o produtor produzir o primeiro som que considera útil, e esse tempo depende do executável escolhido? Sobre curva de aprendizagem: quais dos seis parâmetros o produtor usa sem instrução, e quais ele ignora? Sobre leitura do binário: o produtor percebe diferença audível entre os três executáveis, e essa diferença corresponde a algo que ele consiga nomear? Sobre MIDI: o mapeamento por nota e controlador expande ou restringe o que o produtor consegue fazer? Sobre erro: a mensagem de erro do parser é compreendida sem explicação externa? Sobre preferência: qual dos três executáveis o produtor escolheria para uma produção real, e por quê? Sobre recomendação: o que o produtor mudaria primeiro no instrumento, e o que ele não mudaria mesmo com liberdade total?

5.7 Procedimento de análise

A análise é qualitativa com apoio quantitativo, e a honestidade estatística define seu alcance: os questionários SUS e de atributos são tratados por estatística descritiva, com mediana e dispersão, e não por teste de significância, porque cinco a oito participantes não sustentam inferência e apresentá-la seria desonesto. O diário de sessão é tematizado por duas pessoas da equipe, que codificam as entradas de forma independente, comparam as codificações e resolvem divergência por discussão, e a telemetria é plotada ao longo do tempo, revelando se o produtor encontrou regiões de som que explorou repetidamente ou se o ajuste ficou restrito ao início da sessão. Do conjunto de saem os indicadores principais: o tempo até o primeiro som útil, porque é o que separa um instrumento de uma demonstração, e, no campo qualitativo, o conjunto de parâmetros efetivamente usados sem instrução e o número de executáveis que o produtor não consegue diferenciar sem ouvir, dois números que dizem mais sobre o valor do instrumento do que qualquer média de satisfação.

5.8 Situação atual e conduta ética

O estudo ainda não foi executado, e o motivo é o mesmo que bloqueia T2 e T4, a ausência de estação de trabalho licenciada, condição sem a qual publicar qualquer resultado de usuário seria fabricar dado; o que existe hoje é o desenho, as perguntas e o procedimento, prontos para execução assim que houver host licenciado. A conduta ética segue a Resolução 466/2012 do Conselho Nacional de Saúde, que orienta pesquisas com seres humanos no Brasil, e o protocolo inclui consentimento livre e esclarecido, sigilo dos dados gravados, retirada a qualquer momento e ausência de recompensa ou coerção, com a gravação de áudio, quando necessária à análise do diário, explícita no consentimento e destruída ao fim do prazo de análise.

6. CONCLUSÃO

O percurso confirmou três afirmações, e cada uma delas carrega seu número ou seu teste. A leitura estática verificada de um artefato arbitrário pode acontecer dentro de um instrumento sem expor o host a risco: o parser valida assinatura, número de seções e limites de cada seção, recorta tamanhos declarados que não existem, recusa entrada degenerada com erro tipado e nunca executa o binário, e nos dez casos de borda todos os caminhos terminaram em erro tipado, sem exceção e sem falha de segmentação. O tratamento de sinal, por sua vez, adequou o material bruto para uso musical, pois o janelamento com bordas nulas somado ao filtro de primeira ordem reduz o bin de corrente contínua em pelo menos 40 dB em relação à leitura bruta e leva a energia abaixo de 20 Hz para menos de -60 dBFS, comparando sempre contra o controle negativo, a mesma conversão sem motor e sem condicionamento, que é o que dá sentido aos números. E a disciplina de tempo real mostrou-se imponível por construção e verificável por build: a separação entre o núcleo, que não conhece a interface, e o adaptador, que conhece o framework, torna impossível o acesso do motor ao estado da interface sem criar um caminho novo, e o guard de alocação falha o build se o caminho de áudio alocar, tendo passado em cinco minutos de reprodução sintética.

As limitações fazem parte do resultado, e o texto não as esconde. A validação dinâmica de tempo real, o ensaio de aceite e o estudo com produtores estão bloqueados pela mesma ausência de estação de trabalho licenciada; a ferramenta que detectaria leitura de memória não inicializada não existe para o compilador disponível no Windows, e por isso o defeito dos parâmetros que arrancam errados permanece como hipótese registrada, sem confirmação; e a cobertura de ramo do parser não é medida por falta de ferramenta. Essa franja de incerteza é também o registro do que a arquitetura comprou: a separação entre domínio de interface e domínio de processamento, com fila sem trava e ponteiro cru na fronteira, é o que permitiu ao ensaio de ponta a ponta existir e aos sanitizadores alcançarem a estrutura, e foi por esse caminho que os dois defeitos de maior custo do projeto foram encontrados por teste, não por acidente em sessão de uso.

6.1 Trabalhos futuros

Os próximos passos seguem a ordem do que bloqueia o quê. Obter uma estação de trabalho de áudio licenciada destrava as três etapas pendentes, T2, T4 e o estudo com produtores, e por isso vem primeiro; o formato CLAP, previsto na arquitetura e fora do MVP, dá portabilidade para Linux e macOS sem recompilar o núcleo; o corpus de mutações por inversão de bit transforma o ensaio de robustez em fuzzing de fato; a codificação da entropia da janela de leitura como cor de fundo da célula da grelha espera uma segunda leitura que não dependa só de cor, porque cor de fundo que codifica um número é o primeiro sinal a falhar o critério de uso da cor; e fechar o defeito dos parâmetros exige um depurador com pontos de observação no arranque, a única ferramenta que separaria hipótese de confirmação.

Referências

AUDACITY TEAM. *Import raw data: the manual of the Audacity audio editor*. [S.l.], 2026. Disponível em: <https://manual.audacityteam.org/m/man/importrawdata.html>.

AUDIO DAMAGE. *Quanta 2 granular synthesizer*. 2026. Disponível em: <https://www.audiodamage.com/products/quanta>.

BROOKE, J. Sus: A quick and dirty usability scale. In: *Usability Evaluation in Industry*. London: Taylor and Francis, 1996. p. 189–194.

CASCONE, K. The aesthetics of failure: post-digital tendencies in contemporary computer music. *Computer Music Journal*, v. 24, n. 4, p. 12–18, 2000.

CLAP. *CLever Audio Plugin specification*. 2024. Disponível em: <https://cleveraudio.org/manual/>.

COOPER, S. *Oval: biography*. 2024. Disponível em: <https://www.allmusic.com/artist/oval-mn0000897181>.

HENKE, R. *Granulator II*. 2013. Disponível em: <https://www.roberthenke.com/technology/granulator/>.

IKEDA, R. *datamatics*. 2006. Disponível em: <https://www.ryojiikeda.com/project/datamatics/>.

JUCE. *JUCE: the C++ application framework for audio and music technology*. 2026. Disponível em: <https://juce.com/>.

KLEIDERMACHER DOMINIK; KLEIDERMACHER, M. *Embedded systems security: practical methods for safe and secure software and systems development*. Oxford: Newnes, 2012. ISBN 9780123864785.

LYDA RICHARD A.; HAMROCK, J. K. Using entropy analysis to find encrypted and packed malware. *IEEE Security & Privacy*, v. 5, n. 2, p. 40–45, 2007.

MICROSOFT. *Microsoft PE format specification*. 2024. Disponível em: <https://learn.microsoft.com/en-us/windows/win32/debug/pe-format>.

PIETREK, M. *Peering inside the PE: a tour of the Win32 Portable Executable file format*. 1994. Disponível em: <https://www.winworldpc.com/blog/peering-inside-the-pe/>.

PIETREK, M. *An in-depth look into the Win32 Portable Executable file format*. 2002. Disponível em: <https://learn.microsoft.com/en-us/archive/blogs/mtpeeb/>.

PIRKLE, W. C. *Designing software synthesizer plug-ins in C++: for RackAFX, VST3, and Audio Units*. 2. ed. New York: Focal Press, 2019. ISBN 9781138566760.

RICHARDSON, M. *A glitch in time: how Oval's 1995 ambient masterpiece predicted our digital present*. 2015. Disponível em: <https://pitchfork.com/features/resonant-frequency/9730-a-glitch-in-time-how-ovals-1995-ambient-masterpiece-predicted-our-digital-present>.

ROADS, C. *Microsound*. Cambridge: MIT Press, 2001. ISBN 9780262681684.

SHANNON, C. E. A mathematical theory of communication. *Bell System Technical Journal*, v. 27, n. 3, p. 379–423, 1948.

SIKORSKI MICHAEL; HONIG, A. *Practical malware analysis: the hands-on guide to dissecting malicious software*. San Francisco: No Starch Press, 2012. ISBN 9781593274073.

SMITH, J. O. *Introduction to digital filters: with audio applications*. [S.l.]: W3K Publishing, 2007. ISBN 9780974884073.

STEINBERG. *VST 3 SDK documentation*. 2024. Disponível em: <https://steinbergmedia.github.io/vst3_dev_portal/>.

W3C. *Website Accessibility Conformance Evaluation Methodology (WCAG-EM)*. 2022. Disponível em: <https://www.w3.org/TR/wcag-em/>.

W3C. *Web Content Accessibility Guidelines (WCAG) 2.2*. 2023. Disponível em: <https://www.w3.org/TR/WCAG22/>.

ZÖLZER, U. *DAFX: digital audio effects*. 2. ed. Chichester: Wiley, 2011. ISBN 9780470669504.